RelayOps — Round 2 Build Report

Evidence-backed employee handover · Local Round 2 implementation

Project summary

RelayOps turns authorised project records into an evidence-backed employee handover. Project records are imported, privacy-filtered, stored as source passages, reconstructed into findings, and tied to exact source quotations. Contradictions and missing evidence stay visible. A human reviewer makes the decision; a project lead approves a versioned snapshot before an incoming contributor can see it.

The ready-to-use Project Chronos workspace models the Round 1 fintech migration scenario with 33 synthetic records: six architecture/runbook sources, eighteen GitHub PR/issue records, a Linear task record, four Slack channel records including restricted evidence, and the original cross-source conflict example. The local deterministic workflow needs no user API credentials. Nebius extraction and private-repository import require a configured key/token.

Completed features

- Public product homepage and a working sign-in screen.
- Local Supabase Auth sample accounts for project lead and incoming contributor.
- Five-stage workflow: Connect, Reconstruct, Verify, Review, and Handover.
- Reconstruct presents the four promised views: a cited decision timeline, ownership map, active blockers and risks, and next actions; each finding opens directly in Review.
- Incoming view turns approved decisions/owners, blockers/risks, and next actions into a cited first-seven-days sequence without inventing dates or ownership.
- Project Chronos synthetic workspace persisted in PostgreSQL, with the six-document, eighteen-repository-record, Linear, and four-channel Slack corpus described in Round 1.
- Import Markdown (including Notion page exports), RelayOps JSON, Linear issue JSON, Slack channel/thread JSON, and DiscordChatExporter channel/thread JSON with structural validation, a 50-record import ceiling, bounded file size/count, thread grouping, timestamp/author provenance, clearance preservation, and duplicate checks.
- Flag a GitHub contributor who authored more than 70% of imported commits in the last 14 days when linked docs, tasks, and discussions do not mention them; the finding is marked as missing evidence and cites the contributing commit records.
- Import a public GitHub README, root/docs Markdown files, recent commits and selected commit/PR diffs, issues/comments, pull requests/reviews/conversation comments/code review comments; optional private-repository token is request-only. Pagination and the returned batch are bounded (50 records maximum).
- Redact common API credentials, bearer tokens, private keys, emails, and phone numbers before storage, using Presidio when available and deterministic scanner fallback otherwise.
- Store sources, passages, claims, citations, graph entities/edges, review decisions, approval snapshots, and receipt acknowledgements in PostgreSQL with pgvector support.
- Reconstruct supported facts deterministically; the Nebius Token Factory/LangGraph extractor is integrated to produce structured facts, embeddings, and temporal graph edges. The seeded Chronos sources are now included in the extraction input.
- Validate every persisted citation against the exact stored passage and quoted text. Model output cannot approve or resolve a disagreement.
- Show disagreements and evidence gaps explicitly in Verify.
- Offer category-specific text exit-interview prompts for missing-evidence findings. Save the contributor’s answer as a clearance-tagged source with an exact quote citation; capture never resolves or approves the claim.
- Search embedded source passages by natural-language query with pgvector cosine similarity; the search RPC runs as the signed-in user so project membership and clearance RLS filter matches before content is returned.
- Keep an append-only decision history for each finding, including disposition, note, saved wording, timestamp, and finding version. Human review actions use optimistic claim versions; any changed claim needs a new decision. A heartbeat-backed project lock prevents source import or extraction from racing with review and approval.

- A lead approval gate blocks incomplete handovers. Versioned snapshots are immutable and repeated approval of one revision returns its existing snapshot.
- Contributors receive only an approved snapshot and only data at their clearance level; source text, citations, and related evidence are filtered server-side and protected by RLS.
- Incoming contributor gets the approved first-seven-days sequence, source-linked project findings, and can acknowledge the specific approved snapshot.

Technology and implementation details

- **Web application:** Next.js 16.4.5 App Router, React 19.3.1, TypeScript 5.9.3. Server-rendered routes protect the workspace; server route handlers validate inputs with Zod and use verified Supabase sessions.
- **Database and identity:** local Supabase stack, PostgreSQL 17, Supabase Auth, pgvector, SQL migrations, seeded synthetic users and data. RLS enforces project membership and clearance; server APIs apply matching checks.
- **Python extractor:** FastAPI and Pydantic service with LangGraph orchestration. Presidio and deterministic scanners redact content before model use. Nebius Token Factory uses `Qwen/Qwen3.5-397B-A17B` for extraction and `Qwen/Qwen3-Embedding-8B` for 768-dimensional embeddings; extraction is schema-validated and every citation is matched against the stored source text.
- **GitHub ingestion:** Octokit makes paginated, bounded requests for README/docs, commits and selected file diffs, issues and comments, pull requests and changed-file diffs, reviews, conversation comments, and code review comments. Tokens are used for the request and are not persisted.
- **Orphaned ownership heuristic:** a deterministic 14-day author-share calculation emits an evidence-gap finding above the Round 1 70% threshold only when no imported document, task, or discussion mentions that contributor; the claim cites the author's actual imported commit messages for human investigation.
- **Evidence integrity:** passages are persisted before claims. Citation validation checks passage membership, exact quote containment, and Unicode code-point offsets consistently across Python, TypeScript, and PostgreSQL. Query embeddings and 768-dimensional passage embeddings are compared by a bounded pgvector cosine-similarity RPC. The RPC uses `SECURITY INVOKER`, and its results remain constrained by project/source/passage RLS even if a caller requests a higher clearance. Citationless imported findings become missing evidence. Version checks prevent stale decisions; heartbeat-backed mutation locks and database write-fence triggers serialize ingestion/extraction with review/approval and reject stale mutation tokens. Approval writes an immutable versioned snapshot that is the contributor-facing source of truth.
- **Privacy boundary:** imported content is treated as untrusted data, model prompts prohibit following source instructions, and generated findings require locally verified citations. Models never make approval decisions.

Main data flow

Notion Markdown / RelayOps. Linear. Slack. Discord JSON / GitHub records

```
→ validate and redact

→ persist sources and exact passages

→ deterministic reconstruction or optional Nebius/LangGraph extraction

→ verify citations and preserve conflicts

→ reviewer decisions and lead approval

→ versioned approved snapshot

→ clearance-filtered incoming view and receipt acknowledgement
```

Changes from the Round 1 proposal

The initial Round 2 build temporarily replaced the planned database, server-side access, GitHub ingestion, vector/graph storage, and LLM extractor with browser-local deterministic logic. That did not meet the Round 1 plan. This build restores the planned architecture: Supabase/PostgreSQL and RLS, server-backed persistence, GitHub imports, a privacy-filtering FastAPI/LangGraph Nebius Token Factory extractor, vector/temporal graph data, and human approval. The LLM provider changed from Gemini to Nebius because the team has Nebius Token Factory credit available and chose to use it for the model and embedding workflow. A deterministic path remains as a no-key baseline for judges; it is not presented as a substitute for the planned AI path. General-purpose workspace creation and team invitations are not implemented in Round 2.

Known limitations / what does not work yet

- A live synthetic end-to-end extractor check passed with Nebius: three facts were returned, all citations passed exact-quote validation, and the source received a 768-dimensional embedding. The full 33-source Chronos corpus has not yet been run through Nebius; model latency and spend scale with imported evidence and generated output. The current Token Factory project returned 31 models at catalog check time.
- Private GitHub repository import requires a read-only fine-grained token. The import is bounded to 50 recent records, selected diffs, and root/docs Markdown; it does not mirror the full repository or watch for later changes.
- The static GitHub Pages URL cannot run these APIs. A server-backed Render deploy needs a hosted Supabase project, the Render services and secret configuration, then a live end-to-end judge walkthrough.
- Only the seeded Project Chronos workspace and local demo accounts are supplied. Arbitrary workspaces and invitations are not included.
- Presidio requires the extractor service; when it is unavailable, the API reports that it is using the deterministic Node redaction fallback.
- An interrupted source/extraction request keeps approval blocked until its 15-minute mutation lease expires; retry the source import or extraction to recover and reconcile the project before reviewing.
- Decision history is an append-only application record, not a cryptographically tamper-evident audit. The comparison viewer and PDF export remain Round 3 work.

Three planned Round 3 improvements

1. **Voice exit micro-interview:** let a departing teammate record a short spoken answer to a specific evidence gap, transcribe it with timestamps, and attach the transcript as cited source evidence for human review.
2. **Dependency graph and Day-1 runbook:** visualize blocker/task dependencies and turn approved dependencies and next actions into an interactive first-week checklist for the incoming teammate.
3. **Tamper-evident audit, version diff, and receipt export:** add cryptographic integrity checks to the existing decision history, show source-to-human wording changes side by side, and export the approved handover and receipt as a PDF.

Local judge walkthrough

1. Start Docker Desktop, run `npx supabase start`, then `npx supabase db reset`.
2. Run `./scripts/setup-local-env.ps1` once to write local-only settings to `.env.local`.
3. Start the extractor in one terminal with `./services/extractor/scripts/start.ps1` and the web app in another with `npm run dev`.
4. Open http://localhost:3000, choose **Open workspace**, and sign in as `lead@relayops.local` with password `RelayOpsDemo!2026`.
5. Connect/import records, inspect Reconstruct and Verify, open a missing-evidence finding in Review, capture a text exit-interview answer, and confirm its citation. Make a separate reviewer decision, then check Decision history before lead approval.
6. Sign out; sign in as `contributor@relayops.local` with the same local demo password to see the clearance-filtered approved snapshot and acknowledge receipt.
7. Optional: configure `NEBIUS_API_KEY` in ignored `.env.local`, restart both services, and run Nebius extraction from Reconstruct. Optional: import a public GitHub repository without a token.

Submission links and current status

- Working local app: http://localhost:3000 (available only while local services are running).
- Source repository: https://github.com/karthik-258-code/RelayOps/tree/codex/relayops-round2.
- Public judge link: not deployed yet. Do not submit the former static GitHub Pages URL as the Round 2 application; it cannot execute this server-backed build.